

Assignment 1: Multi-Core and SIMD Parallelism

Saad | Roll Number: <roll number> | BS(CS)-7G

1. Overview and Test Setup

This report covers all three programs of Assignment 1. Part 1 parallelises a compute-bound cubic fractal renderer across threads, Part 2 vectorises an irregular scalar loop against the simulated SIMD instruction set in PDCvector.h, and Part 3 parallelises the memory-bandwidth-bound AXPBY kernel. Every number quoted below was measured on the machine described in this section, using the timing harnesses supplied with the starter code.

All measurements were taken on a 16-logical-core x86-64 machine running Linux under WSL2. Each program was compiled at -O2 with C++11/C++17 and linked against pthreads, exactly as the provided Makefiles specify. Because wall-clock timing is noisy, every sweep was repeated several times and the best (or clearly typical) run was recorded, as the assignment tips recommend. Correctness was re-checked with the --check flag after every code change; no timing result is reported for a configuration that did not first pass its correctness test.

- Part 1: ./fractal -t N, --view 1|2, --sweep, --check (900x601 image, maxIterations = 300).
- Part 2: ./altseries -s N, -b for the bonus, with VECTOR_WIDTH set to 2, 4, 8 and 16 at compile time.
- Part 3: ./axpby -n N, -t N, --sweep, --check (alpha = 2.5, beta = -1.5).

2. Part 1: Parallel Fractal Rendering with Threads

2.1 Workload and why no synchronisation is needed

The renderer walks a 900x601 grid of pixels. Each pixel is mapped to a complex number c and the recurrence $z \leftarrow z^3 + c$ is iterated until the magnitude escapes a fixed radius or until the iteration budget of 300 is exhausted. The returned iteration count is the pixel value, so bright pixels are literally the pixels that consumed the most arithmetic. This makes the workload compute-bound, but it also makes the cost per pixel, and therefore the cost per image row, highly non-uniform.

The parallel decomposition assigns whole rows to threads. Because a row is owned by exactly one thread, and the output buffer is indexed as `output[row * width + col]`, no two threads ever write to the same address. Disjoint ownership removes the data race entirely, which is why the implementation needs no mutex, atomic, or barrier of any kind. All student code lives in `fractalWorker()`; the supplied `fractalThread()` spawns workers 1..N-1, runs worker 0 on the calling thread, and then joins.

2.2 Stage 1: two-thread spatial decomposition

The first stage splits the image into two horizontal halves: thread 0 renders rows 0 to 299 and thread 1 renders rows 300 to 600. Note that the height, 601, is odd, so the two halves are deliberately unequal in size (300 rows against 301 rows) rather than dropping the middle row. Running `./fractal -t 2 --check` reported a pixel-identical result against the serial reference and a speedup of 2.01x on View 1, confirming that the threading path itself was sound before any load-balancing work began.

2.3 Stage 2: contiguous blocks for one to eight threads

The two-half split generalises to any thread count by giving each thread one unbroken block of rows. The block boundaries are computed as `startRow = (height * threadId) / numThreads` and `endRow = (height * (threadId + 1) / numThreads)`.

$(\text{threadId} + 1) / \text{numThreads}$. This multiply-then-divide form is important: it distributes the remainder rows automatically, so with 601 rows and 3 threads the blocks are [0, 200), [200, 400) and [400, 601). Every row is rendered exactly once, and no row is rendered twice, for any thread count from 1 to 16.

Table 1. Naive contiguous-block speedup from `./fractal --sweep --check`.

Threads	View 1 speedup	View 2 speedup
1	0.99	1.00
2	1.96	1.16
3	2.26	1.79
4	2.97	2.20
5	2.92	2.74
6	4.40	3.28
7	5.11	3.43
8	5.62	4.04

Speedup is clearly not linear. At eight threads the naive mapping delivers only 5.62x on View 1 and 4.04x on View 2, well short of the ideal 8x. The reason is that equal numbers of rows do not represent equal amounts of work: a thread that happens to own the dense part of the fractal performs far more iterations than a thread that owns a mostly-escaped region, and the whole image is not finished until the slowest thread finishes.

2.4 The three-thread anomaly on both views

The three-thread datapoint is the clearest illustration of this effect, and it fails for a different reason on each view. Because the total run time is bounded below by the slowest thread, the per-thread timings collected in stage 3 explain the anomaly directly.

Table 2. Per-thread work time, naive contiguous mapping, three threads.

Thread	Rows owned	View 1 time (s)	View 2 time (s)
0	0 - 199	0.1058	0.1428
1	200 - 399	0.1526	0.1100
2	400 - 600	0.1082	0.0064

On View 1 the fractal body sits in the vertical centre of the frame, so thread 1 owns the expensive middle band and takes 0.1526 s while its two neighbours finish in roughly 0.108 s. The run time is set by thread 1, which is why the measured speedup is only about 2.3x instead of 3x: two of the three threads spend approximately thirty percent of the run idle.

On View 2 the imbalance is far more severe and sits at the opposite end of the image. View 2 is a zoom into the region x in $[-0.90, -0.30]$, y in $[0.30, 0.90]$, so nearly all of the expensive pixels live in the upper part of the frame. Thread 2, which owns the bottom third, finishes in 0.0064 s, roughly twenty-two times faster than thread 0. One of the three threads is therefore effectively unused for the entire render, and the speedup collapses to about 1.8x. The important conclusion is that a single static contiguous policy cannot be tuned for both views, because the vertical cost distribution is different in each.

2.5 Stage 4: interleaved row assignment

The fix keeps the number of rows per thread essentially unchanged and instead changes which rows each thread owns. Thread i is assigned rows i , $i + \text{numThreads}$, $i + 2 * \text{numThreads}$, and so on, so each thread's rows are spread uniformly over the whole image rather than clustered into one band. Adjacent expensive rows are now distributed round-robin across all threads, so every thread receives a statistically similar mixture of cheap and expensive work. The policy is a single static rule that is applied identically at every thread count, it requires no synchronisation, and it is selected at compile time by `FRACTAL_INTERLEAVE` (0 for the naive mapping, 1 for the interleaved mapping) so that both versions remain available in the submitted source.

Table 3. Interleaved-row speedup from `./fractal --sweep --check` (best of runs).

Threads	View 1 speedup	View 2 speedup
1	0.99	1.01
2	1.95	2.00
3	2.88	2.92
4	3.81	3.80
5	4.70	4.53
6	5.63	5.68
7	5.45	5.37
8	7.56	7.41

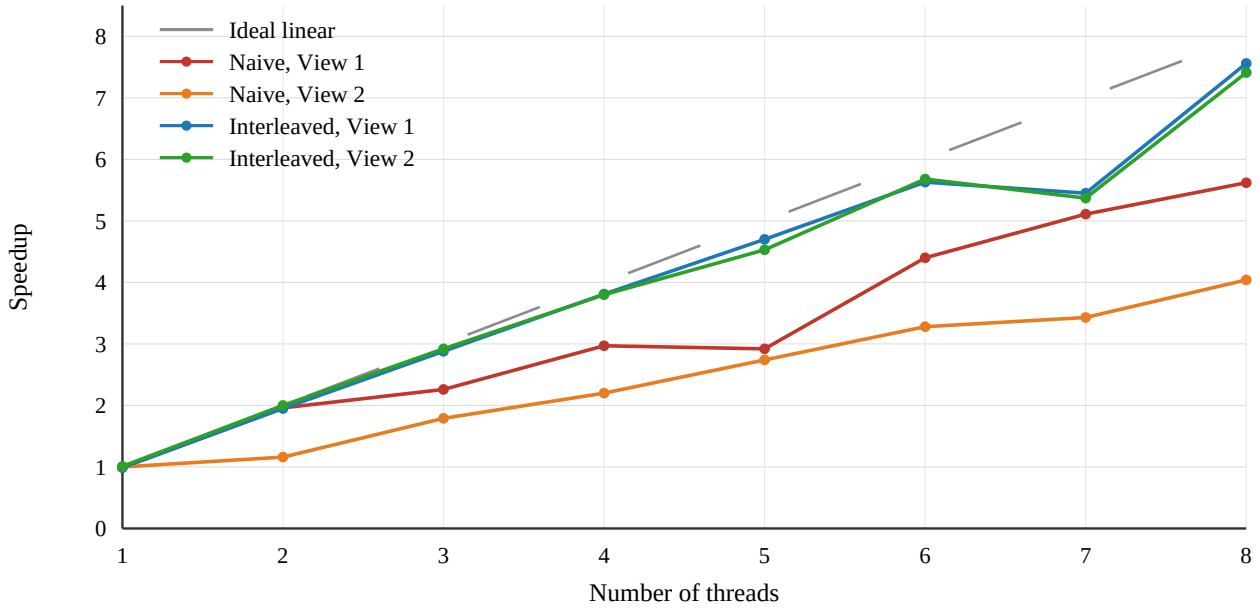


Figure 1. Fractal speedup against thread count for both views, naive contiguous blocks versus interleaved rows.

The interleaved mapping reaches 7.56x on View 1 and 7.41x on View 2 at eight threads, both inside the seven-to-eight-times target, and it removes the three-thread anomaly on both views (2.88x and 2.92x, against 2.26x and 1.79x before). The per-thread timings confirm the mechanism rather than merely the outcome: at three threads on View 1 the three workers now take 0.1235 s, 0.1235 s and 0.1208 s, and at eight threads on View 1 they span only 0.0483 s to 0.0575 s. Compared with the naive eight-thread spread of 0.0238 s to 0.0681 s, the gap between the fastest and slowest thread has shrunk from nearly threefold to about twenty percent.

Table 4. Load balance before and after the mapping change.

Configuration	Fastest thread (s)	Slowest thread (s)	Spread
Naive, 8 threads, View 1	0.0238	0.0681	2.86x
Interleaved, 8 threads, View 1	0.0483	0.0575	1.19x
Interleaved, 8 threads, View 2	0.0335	0.0465	1.39x

2.6 Stage 5: sixteen threads

Running the interleaved build at sixteen threads gave no meaningful improvement over a well-balanced eight threads: a representative run measured 7.14x on View 1 and 6.26x on View 2 at sixteen threads, which is within run-to-run noise of the 7.56x and 7.41x already obtained at eight. This is the expected result. The host exposes sixteen logical cores but far fewer independent floating-point pipelines, so once the work is evenly divided the extra threads compete for the same execution resources and add scheduling and cache-contention

overhead instead of additional throughput. Extra threads only help while there is genuinely idle hardware for them to occupy, and after stage 4 there is not.

3. Part 2: SIMD Vectorisation of the Clamped Alternating Series

3.1 The algorithm and why it resists vectorisation

For each element the serial reference starts with `acc = values[i]` and then takes up to `counts[i]` steps, multiplying by `values[i]` on even steps and adding `values[i]` on odd steps. As soon as `acc` exceeds `caps[i]` the accumulator is replaced by the cap and that element stops early. Finally any result below 0.0001 is snapped to exactly zero. The difficulty for SIMD is that both the trip count and the early exit are per-element: within a single vector, one lane may finish after two steps while another still has twenty-four steps remaining.

3.2 Mask strategy

The implementation processes the array in chunks of `VECTOR_WIDTH` and maintains two distinct masks, exactly as the assignment requires. The first, `valid`, is built with `_pdc_init_first_n()` and marks the lanes that correspond to real array elements; it is what makes the final partial chunk safe when `N` is not a multiple of the vector width, and it is the gap deliberately left in the worked `absVector()` example. The second, `alive`, tracks the lanes that have not yet clamped. On every step the two are combined with the `still-has-steps` predicate, obtained as `_pdc_vgt_int(counts, j)` because the instruction set provides no integer less-than, to form the work mask that gates the multiply or add.

Clamping is handled in two moves. A comparison against `caps` produces the mask of lanes that just crossed their limit; `_pdc_vmove_float()` writes the cap into those lanes, and `_pdc_mask_not(hitCap, alive)` then clears them from `alive` permanently. Because unmasked lanes of every arithmetic instruction retain their previous value, a cleared lane is frozen for the rest of the chunk even though its own step budget might still say it is active. The loop terminates as soon as the work mask is empty, checked with `_pdc_cntbits()`. Results matched the serial reference for `N = 10000`, for `N = 10003` and for `N = 7`, which exercises both the tail path and the case where the array is smaller than one vector.

3.3 Vector utilisation against vector width

Table 5. Reported utilisation for `.altseries -s 10000` at each vector width.

VECTOR_WIDTH	Vector utilisation
2	67.6%
4	62.9%
8	59.1%
16	56.2%

Utilisation decreases monotonically as the vector width grows, falling from 67.6 percent at width 2 to 56.2 percent at width 16. The cause is that all lanes of a vector advance in lockstep, so the chunk cannot retire until its last busy lane finishes. Every lane that has already clamped, or that had a small `counts[i]` to begin with, is still issued as part of every subsequent instruction but contributes nothing; the hardware pays for the full width while only the surviving lanes do useful work.

Widening the vector makes this worse because it groups more independent elements under one shared exit condition. At width 2 a chunk only needs its single partner lane to keep going, whereas at width 16 the chunk keeps stepping until the longest-running of sixteen elements is done, and by that point most of the other fifteen lanes have long since been masked off. In other words the number of wasted lane-slots per chunk grows faster than the useful work does, so the measured utilisation falls even though the total instruction count drops. This is the classic penalty of divergent control flow on wide SIMD units.

3.4 Bonus: vectorised dot product

The optional `dotProductVector()` was implemented as well. Each chunk loads both operands, multiplies them lane-wise, and then reduces the product vector using $\log_2(\text{VECTOR_WIDTH})$ rounds of `_pdc_hadd_float()` followed by `_pdc_interleave_float()`: the horizontal add sums lane pairs, and the interleave repacks those pair sums so that the next horizontal add can combine them again. The reduction therefore costs a logarithmic number of vector instructions per chunk rather than a linear scan over the lanes. Verified with `./altseries -s 10000 -b`, the result matched the serial dot product at every tested width.

4. Part 3: AXPBY, a Memory-Bandwidth-Bound Workload

4.1 Contiguous static decomposition

AXPBY computes $\text{result}[i] = \alpha * X[i] + \beta * Y[i]$ with $\alpha = 2.5$ and $\beta = -1.5$. Part 3 requires a contiguous decomposition rather than the interleaved policy that won Part 1: thread t receives one unbroken index range, computed as $\text{start} = (N * \text{threadId}) / \text{numThreads}$ and $\text{end} = (N * (\text{threadId} + 1)) / \text{numThreads}$. As in Part 1 this form absorbs the remainder automatically, so no index is dropped or computed twice when N is not divisible by the thread count. Contiguous ranges are the right choice here because each thread then streams sequentially through memory, which is what the hardware prefetchers expect; an interleaved mapping would have every thread touching every cache line. Each thread writes a disjoint slice of result, so again no mutex is required.

4.2 Correctness across several array sizes

Correctness was verified at three different problem sizes, including one that is not a multiple of 64, to prove that the remainder handling is right rather than merely lucky.

Table 6. Correctness of the threaded AXPBY implementation for uneven N .

Array size N	Divisible by 64	Threads	--check result
1,000,003	no	4 and 8	PASSED
8,000,001	no	3 and 5	PASSED
20,000,000	no	4 and sweep 1-8	PASSED

4.3 Measured scaling

Table 7. `./axpby -n 20000000 --sweep --check`.

Threads	Time (s)	Bandwidth (GB/s)	Speedup
1	0.0121	26.482	1.05
2	0.0092	34.678	1.37
3	0.0095	33.512	1.33
4	0.0097	32.977	1.30
5	0.0090	35.436	1.40
6	0.0094	34.156	1.35
7	0.0090	35.379	1.40
8	0.0090	35.646	1.41

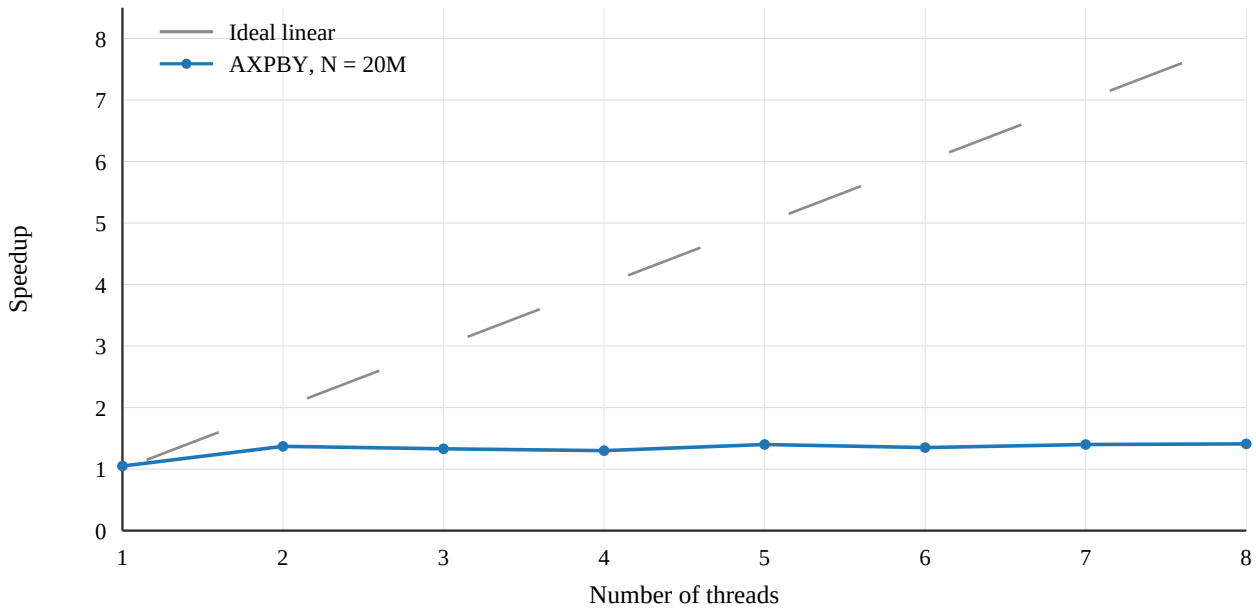


Figure 2. AXPBY speedup against thread count, compared with ideal linear scaling.

The curve flattens almost immediately. Two threads already reach 1.37x, and every count from two to eight stays in a narrow band around 1.3x to 1.4x while the achieved bandwidth saturates in the mid-thirties of gigabytes per second. Adding threads beyond the second buys essentially nothing, because the limiting resource is not arithmetic throughput but the rate at which the memory system can supply and retire data.

4.4 Compute-bound against memory-bound: the role of arithmetic intensity

Comparing the two speedup curves makes the distinction concrete. The fractal renderer scales to roughly 7.5x at eight threads once its load is balanced, while AXPBY stalls near 1.4x. The deciding factor is arithmetic intensity, the ratio of arithmetic performed to bytes moved. The fractal performs up to 300 iterations of several multiplies and adds on a handful of registers per pixel, so its operands stay in registers and each core can be kept busy independently; adding cores adds usable floating-point throughput. AXPBY performs one multiply and one add per element and then must fetch the next element, so its intensity is only about 0.17 floating-point operations per byte. A couple of threads are enough to reach the bandwidth ceiling, and every additional thread simply queues behind the same shared memory path. This is why the fractal scales better, and why more threads are the wrong lever for a bandwidth-bound kernel.

The harness also reports traffic as $4 * N * \text{sizeof}(\text{float})$ even though the source shows only two reads and one write per element, which would suggest three floats. The extra factor comes from write-allocate behaviour in the cache hierarchy. Storing to `result[i]` does not write a lone float to memory; it writes into a cache line, and because that line is not already resident it must first be read from memory before the partial write can be merged into it. The result array is therefore transferred twice, once implicitly on the read-for-ownership and once when the dirty line is eventually evicted, giving two floats for X and Y plus two for result, or four floats per element in total. A non-temporal streaming store would avoid the extra read and bring the count back down to three.

5. Summary

- Part 1: the threaded renderer is pixel-identical to the serial reference on both views. The naive contiguous mapping is limited by load imbalance, diagnosed at three threads with per-thread timings; the interleaved mapping reaches 7.56x on View 1 and 7.41x on View 2 at eight threads with no synchronisation.
- Part 2: `clampedAltVector()` matches the serial output for any N and any vector width, including tails, using separate validity and liveness masks. Utilisation falls from 67.6 percent at width 2 to 56.2 percent at width 16 because divergent lanes idle while their neighbours continue. The bonus dot product reduction was also implemented in logarithmic vector instructions.
- Part 3: the contiguous AXPBY decomposition passes `--check` for uneven N and saturates near 1.4x, confirming that a low-arithmetic-intensity kernel is limited by memory bandwidth rather than by core count.